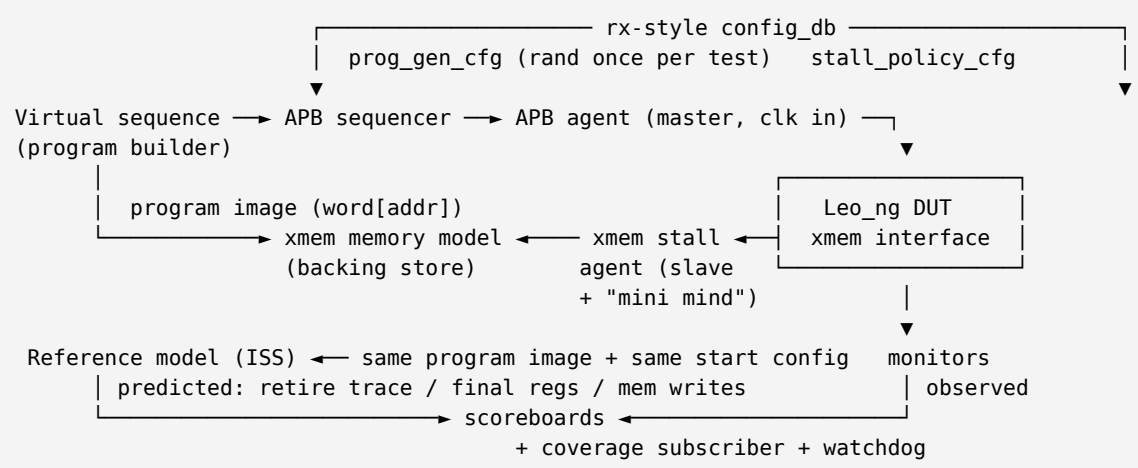
Leo_ng Controller — Verification Plan (draft v0.1)

Modelled on the *RX DMA Verification Plan* (agents / sequences / reference model / scoreboards / coverage), with the stimulus built by the *Termination-Aware Random Program Generator* method.

> **Assumptions to confirm** (I have no RTL/spec in the repo yet — each is marked `[?]` where it matters) - The controller is a CPU-like core that fetches instructions and does loads/stores through an **external memory (xmem)** interface with a wait/ready handshake. - **APB** is the control/status path: reset-vector/config, start, halt/status, interrupts/error regs. - The **clock is an input** to the APB agent (a virtual interface handle). The agent never generates it; a separate clock/reset driver in the TB top does. - A golden **ISS / reference model** exists or will be written.

---

1. Environment overview



Component	Role
APB agent	Master. Programs the DUT (start address, enables), starts it, polls/receives status & halt. Takes `clk` (and `rst_n`) through its virtual interface. Monitor always active.
XMEM stall agent	Slave responder on the instruction/data memory port. Serves reads/writes from the backing memory model and decides *per access* how many wait states to insert.
Virtual sequence	Builds a **terminating random program**, hands the image to the memory model, then drives APB start/wait/check. Detailed in §3.
Reference model	Runs the same program on the same initial state; produces expected architectural results.
Scoreboards	Retire/PC trace, register file at halt, xmem write stream, APB status/error registers.
Watchdog	TB-side timeout with a *computed* bound (§3.7). Catches DUT deadlock — not generator bugs.

---

2. Agents

2.1 APB agent (master, clock as input)

- Same pattern as the RX `dv_agent`: sequencer + driver + monitor, `is_master` toggles driver/sequencer.
- `clk`/`rst_n` come in via the virtual interface; the driver only waits on `@(posedge vif.pclk)`. This keeps one clock owner in the TB top and lets the same agent run at any clock ratio to the core clock.
- Item: `apb_item {addr, data, write, wait_cycles}`; optional random inter-transfer gaps (like `valid_probability` in RX).
- Used for: program-start config, `START`, `HALT`/`DONE` polling, reading result/error registers, and mid-run disturbance (e.g. reading status while the core runs).

2.2 XMEM stall agent (slave with a “mini mind”)

It is *reactive* — no sequencer needed to drive transfers, but it owns a small **stall policy** object that decides when to hold the controller off.

Per-access decision: `wait_states = policy.decide(access)` where `access = {addr, is_fetch, is_write, is_branch_target, back_to_back_count}`.

Policy mode	Behaviour	What it hunts
`NONE`	0 wait states	Max-throughput baseline
`RANDOM_LOW` / `RANDOM_HIGH`	Random wait with probability *p* (like RX `valid_probability`: 100 / 25 / 1-in-4)	General handshake robustness
`BURSTY`	Quiet for *N* accesses, then a long stall	Pipeline-drain / restart bugs
`TARGETED`	Stall exactly on: first fetch after a taken branch, load immediately followed by use, store followed by load of same address, last instruction before `PROGRAM_END`	Hazards and corner timing the program actually contains
`STARVE`	One very long stall, still under the watchdog bound	Timeout/watchdog logic in DUT

The “mini mind” is just: **mode + a small history window + a seeded RNG**, re-picked every *K* accesses (a phase), so one test visits several modes. It may read the program’s metadata (addresses of loop headers, back-edges, indirect jumps — published by the virtual sequence through config_db / a shared `program_map`) to place stalls where the program is interesting. Keep it deterministic per seed so failures replay.

The agent also provides the **backing memory model**: `mem[addr]` preloaded with the program image + data region; a backdoor `load_image()` and a monitor that logs every access to the analysis port.

---

3. The virtual sequence — program builder (the hard part)

3.1 Why it is not plain `randomize()`

“Does this program terminate?” is a **graph property** (no unguarded cycle). A SystemVerilog constraint solver cannot express reachability/cycle structure over a free-form instruction array. So the sequence **constructs the program in stages** inside `body()`, using `randomize()` only for local decisions (how many blocks, which edge, which opcode). Same idea as the RX plan: generation parameters are randomized once, then the sequence executes a fixed *algorithm*.

3.2 Items it produces

```
prog_gen_cfg    (rand once/test)   num_blocks, loop_prob, back_edge_prob, indirect_jmp_prob,
                                  max_loop_guard, max_nest_depth, instr_mix weights, mem region, ...
cfg_node        one basic block: id, succ[] (edges), kind {NORMAL, LOOP_HEADER, GUARD, ENTRY, END}
bb_item         one block filled with instructions: instr[], terminator derived from succ[]
prog_item       the whole program: bb_item[], image[addr]->word, metadata (loop headers,
                guard regs, indirect target sets, instruction/dynamic-instruction upper bound)
```

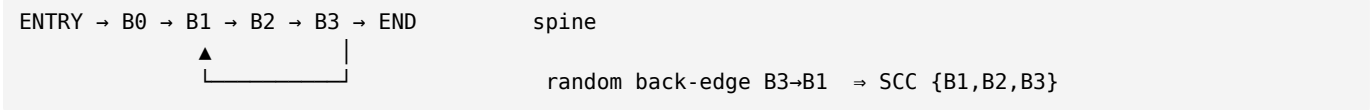
3.3 Pipeline (one stage = one method of the sequence)

```
 build_cfg() → analyze_loops() → insert_guards() → alloc_registers() → fill_blocks() → validate() →
emit()
```

#	Stage	What it does	Why
1	**build_cfg**	Create `ENTRY`, `END`, *N* blocks. First lay a **spine** `ENTRY→B0→…→Bn→END` (guarantees END is reachable). Then add random extras: conditional forward branches, **backward edges** (loops), indirect-jump nodes with a pre-chosen **legal target set** `{B3,B8,B12}` and an edge to each.	Control flow decided *before* instructions, so it can be analysed.
2	**analyze_loops**	Tarjan **SCC** on the CFG. Every SCC with >1 node (or a self-loop) is a cycle that could run forever. Record its header and member set; compute nesting.	Finds every place an infinite run is possible — including cycles formed by indirect-jump edges.
3	**insert_guards**	For each SCC: a counter register `rG` initialised in the pre-header to a random budget (1..`max_loop_guard`); in the header add `rG -= 1; if rG==0 → exit edge`. Exit edge goes out of the SCC (or to `END`). The guard test comes **first**, so no data-dependent random branch can keep the loop alive.	Turns “maybe infinite” into “provably bounded”, yet loops still run many iterations (still useful stimulus).

#	Stage	What it does	Why
4	**alloc_registers**	Register manager with states `FREE / CONTROL_FLOW / LOOP_GUARD / RESERVED`. Pick guard regs and indirect-target regs *first*, mark them protected; random instructions may only write `FREE` regs.	Stops a random `ADD r7,…` from clobbering a loop counter or jump target.
5	**fill_blocks**	For each block fill random ALU / load / store / dependent-chain instructions into `FREE` regs (load/store addresses constrained into the data region). The **last instruction is derived from the CFG edges** (fall-through, `BNE r9,r0,B2`, `JMP r7`), never random.	Instructions can be as random as you like; control flow is already safe.
6	**validate**	Check: every branch/jump target is a legal block; no protected reg written by random code; every SCC has a guard on every cycle path; `END` reachable from every block; compute the **dynamic instruction bound** (§3.7). Fail ⇒ `uvm_fatal` (generator bug, not DUT bug).	Cheap proof before simulating anything.
7	**emit**	Linearise blocks → addresses → words; place `PROGRAM_END` (halt) at `END`; publish `prog_item` to the memory model (backdoor `load_image`) and metadata to the reference model, stall agent, coverage via config_db.	One source of truth for DUT, ISS and stall policy.

3.4 Worked example (matches the PDF figure)



`analyze_loops` finds SCC {B1,B2,B3}, header B1. `insert_guards` adds in the pre-header (B0) `LI r20, 7` and in B1 `SUBI r20,r20,1 ; BEQZ r20, END`. Whatever the random branch in B3 does, B1 runs at most 7 times, then control leaves to `END`. `r20` is `LOOP_GUARD`, so `fill_blocks` never writes it.

3.5 Indirect jumps

`JMP r7` is only legal if `r7` ∈ the pre-declared set `{B3,B8,B12}`. The CFG already has an edge to each, so SCC analysis already accounts for them. `r7` is `CONTROL_FLOW`: random code can't touch it; the block just before the jump loads it from a randomly selected member of the set.

3.6 Test flow (mirrors RX `rx_virtual_sequence` phases)

Phase	RX DMA equivalent	Leo_ng
0	generation params via config_db	Randomise `prog_gen_cfg` + `stall_policy_cfg` once, publish
1	initial junk	Optionally pre-fill memory with junk/illegal words outside the program, and let APB do a few harmless reads → DUT must not depend on clean memory
2	lock acquisition	**Build program** (stages 1-7), program start address over APB
3	test body (`REGULAR` / `REAL`)	`START` via APB, wait for `HALT/DONE` (or watchdog), read results. `REGULAR`: one random program. `REAL`: loop of many programs/transactions back-to-back, resetting or re-starting the DUT between them
4	end of test	Final-state compare, coverage sample, drain objection

Suggested test types (like `lock_type`): `SPINE_ONLY` (no loops), `SINGLE_LOOP`, `NESTED_LOOPS`, `INDIRECT_HEAVY`, `MEM_HEAVY` (load/store dense), `TIGHT_LOOP` (1–2 block loops to hammer the stall agent's branch-target case).

3.7 Termination & watchdog numbers

`validate()` computes `max_dyn_instr` = Σ over blocks of (block_len × product of enclosing loop guards). Then

```
watchdog_cycles = max_dyn_instr × (1 + max_stall_per_access × accesses_per_instr) + margin
```

So a hang is either a **generator bug** (validate would have caught it) or a real **DUT bug** — the watchdog distinguishes them. Keep `max_nest_depth` small (≈3): nested guards multiply.

3.8 Open design choices `[?]`

1. **How the image reaches memory**: backdoor `load_image()` (fast, recommended) vs. writing it through APB/xmem (exercises more of the DUT, much slower). Could do backdoor by default, front-door in a dedicated test.
2. **Self-modifying code / stores into the program region**: forbid in v1 (data stores constrained to a separate data region).
3. **Exceptions/interrupts**: out of scope for v1 unless the spec requires them; add an exception handler block to the CFG later.

---

4. Reference model & scoreboards

- Reference model (ISS) takes `prog_item.image` + start config, runs to `PROGRAM_END` using the **same instruction bound**, emits: retired-instruction trace (PC, rd, value), final register file, memory write stream.
- Scoreboards: **trace** (PC/rd/value per retire — pinpoints first divergence), **memory** (xmem writes in order, from the stall agent's monitor), **status** (APB DONE/error registers).
- Stalls must change *timing only*, never results: the scoreboards are timing-agnostic, so any mismatch under stall ⇒ DUT bug.

5. Coverage (initial list)

- CFG shape: loop count, nesting depth, SCC size, indirect-jump fan-out, back-edge vs. forward.
- Instruction mix and **hazard pairs** (load→use, ALU→branch, store→load same addr).
- Stall policy × event cross: {each mode} × {stall on branch target, load, store, last fetch, back-to-back}.
- Guard expiry: loop exits by guard vs. by data condition.
- APB: read-while-running, back-to-back transfers, inter-transfer gaps.

6. Milestones

1. APB agent + clock/reset plumbing; stall agent with `NONE`/`RANDOM` modes; directed program (no generator).
2. Generator stages 1–3 + 6–7 (spine + guarded loops, ALU only) + ISS trace compare.
3. Register manager + load/store + indirect jumps.
4. Stall "mini mind" `TARGETED`/`BURSTY` using program metadata.
5. Coverage closure, regression with seed sweep.